AED2 - Atividade Prática - Aula 4

Aluno: João Marcelo Franca Batista

1) Radix Sort de datas: sub-algoritmo estável x instável

Especificações da máquina usada no teste

Processador: Intel(R) Core(TM) i5-1035G1 @ 1.00GHz (4 núcleos / 8 threads)
Memória RAM: 8 GB
Sistema operacional: Windows 11 Home Single Language (64-bit)
Compilador: gcc 13.2.0 (MinGW-w64 / MSYS2), sem otimização (-O0)

Estratégia de armazenamento e do Radix Sort

Cada data foi armazenada numa struct (TAD) com os campos dia, mês e ano, para que os 3 valores de uma mesma data sempre andem juntos. O vetor tem 1000 datas aleatórias (dia 1-31, mês 1-12, ano 2000-2024), seed fixa (srand(42)). O Radix Sort foi feito em 3 passadas (dia, mês, ano), cada uma testada de duas formas: com Counting Sort (estável) e com Selection Sort (instável) por campo.

Tempo de execução e efetividade

Radix Sort com Counting Sort (ESTÁVEL):
  Corretamente ordenado? SIM
  Tempo de execução: 0.000042 s

Radix Sort com Selection Sort (INSTÁVEL):
  Corretamente ordenado? NÃO
  Tempo de execução: 0.004171 s

Resultado: a versão ESTÁVEL não só foi muito mais rápida (~100x), como foi a única que produziu o vetor corretamente ordenado por (ano, mês, dia). A versão INSTÁVEL embaralha datas do mesmo mês/ano que já estavam certas numa passada anterior — por exemplo, apareceram as datas 02/04, 02/04 e 01/04 fora de ordem no resultado final.

Análise de complexidade de tempo

Notação usada: $O(f(n))$ para o pior caso, $\Theta(f(n))$ para o caso médio e $\Omega(f(n))$ para o melhor caso.
O Counting Sort não compara elementos entre si: ele conta quantas vezes cada valor aparece (o campo só varia entre poucos valores possíveis, ex.: 1 a 31 no dia) e usa essa contagem para já colocar cada elemento na posição certa. Isso custa $O(n)$ e não depende da ordem de entrada: é $O(n)$, $\Theta(n)$ e $\Omega(n)$ nos três casos. Como o Radix Sort só repete isso 3 vezes, continua $O(n)$ no total.
O Selection Sort sempre percorre o vetor inteiro procurando o menor valor restante, não importa se os dados já estão ordenados ou não. Por isso é $O(n^2)$, $\Theta(n^2)$ e $\Omega(n^2)$ nos três casos, mesmo sendo usado só dentro de cada passada do Radix Sort.
Trocar o Counting Sort ($O(n)$) pelo Selection Sort ($O(n^2)$) deixa o Radix Sort muito mais lento e ainda tira a garantia de estabilidade entre as passadas, fazendo o resultado final sair errado.